INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-04: Clases Objetos y Métodos
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Alulema Chato Sandra Jazmin
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Rubén Caiza Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:
Desarrollar programas utilizando clases, objetos y métodos, lo cual permite disminuir errores y reutilizar el código.

Específicos:

- Implementar clases con atributos, constructores y métodos que usen el tipo de retorno correcto en cada firma.
- Aplicar objetos como atributos y como parámetros de métodos para lograr la colaboración entre clases.
- Validar condiciones (stock, fondos, disponibilidad) dentro de los métodos para mantener consistente el estado de los objetos.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6 horas
No presenciales: 0 horas

2.4 Instrucciones

La tarea se realizó de forma individual.
Se leyó detenidamente cada uno de los ejercicios planteados en el aula virtual y se desarrolló lo solicitado.
Se codificó en Java cada uno de los ejercicios planteados.
Al finalizar, se comprime y sube al aula virtual el proyecto desarrollado; adicionalmente, se realizan capturas de cada uno de los ejercicios, las cuales se suben en un archivo PDF en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Computador o laptop
- Visual Studio Code con Java (JDK)
- Plataforma educativa de la UTA (aula virtual)
- Internet para consultas
- Papel y material de escritorio

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☐Simuladores y laboratorios virtuales

☐Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial
Otros (Especifique): Visual Studio Code (editor para programar en Java)

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas: resolver en Java los cinco ejercicios de integración de clases, objetos y métodos.

2.7 Resultados obtenidos

Ejercicio 1: Libro y Autor

Se crearon las clases Autor y Libro. Libro contiene un atributo de tipo Autor (objeto como atributo) y su método mostrarInformacion() imprime el título, el año y los datos del autor.

- En main se crearon dos libros.

Autor.java

```
public class Autor {
    String nombre;
    String nacionalidad;

    public Autor(String nombre, String nacionalidad) {
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }
}
```

Libro.java

```
public class Libro {
    String titulo;
    int anio;
    Autor autor; // objeto como atributo

    public Libro(String titulo, int anio, Autor autor) {
        this.titulo = titulo;
        this.anio = anio;
        this.autor = autor;
    }

    public void mostrarInformacion() {
        System.out.println("Título: " + titulo);
        System.out.println("Año: " + anio);
        System.out.println("Autor: " + autor.nombre
                + " (" + autor.nacionalidad + ")");
        System.out.println("--------------------------");
    }
```

```
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        Autor a1 = new Autor("Edgar Allan Poe", "Estadounidense");
        Autor a2 = new Autor("Jorge Icaza", "Ecuatoriano");

        Libro l1 = new Libro("El gato negro", 1843, a1);
        Libro l2 = new Libro("Huasipungo", 1934, a2);

        l1.mostrarInformacion();
        l2.mostrarInformacion();
    }
}
```

.

Resultado en consola:

```
Problems 1  Debug Console  Output  Terminal  Ports

PS C:\Users\VICTUS\OneDrive\guiaape4>
PS C:\Users\VICTUS\OneDrive\guiaape4>  c:; cd 'c:\Users\VICTUS\OneDrive\guiaape4'; & 'C:\Users\VICTUS
\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\VICTUS\AppData\Roaming\Code\User\workspaceStorage\bbff3
4f8966407b46a0f2b3e9759bbb4\redhat.java\jdt_ws\guiaape4_6f488561\bin' 'Main'
Título: El gato negro
Año: 1843
Autor: Edgar Allan Poe (Estadounidense)
---------------------------
Título: Huasipungo
Año: 1934
Autor: Jorge Icaza (Ecuatoriano)
---------------------------
PS C:\Users\VICTUS\OneDrive\guiaape4>
```

Ejercicio 2: Rectángulo

La clase Rectangulo implementa calcularArea() y calcularPerimetro() (double), esCuadrado() (boolean), obtenerDescripcion() (String) y mostrar() (void), cada uno con el tipo de retorno que le corresponde.

- Se probó con un rectángulo de 4.0 x 6.0 y un cuadrado de 5.0 x 5.0.

Rectangulo.java

```
public class Rectangulo {
    double base;
    double altura;

    public Rectangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }
```

```
    public double calcularArea() {
        return base * altura;
    }

    public double calcularPerimetro() {
        return 2 * (base + altura);
    }

    public boolean esCuadrado() {
        return base == altura;
    }

    public String obtenerDescripcion() {
        return "Rectángulo de " + base + " x " + altura;
    }

    public void mostrar() {
        System.out.println(obtenerDescripcion());
        System.out.println("Área: " + calcularArea());
        System.out.println("Perímetro: " + calcularPerimetro());
        System.out.println("¿Es cuadrado? " + (esCuadrado() ? "Sí" :
"No"));
        System.out.println("---------------------------");
    }
}
```

Main.java

```
public class Main2 {
    public static void main(String[] args) {
        Rectangulo r1 = new Rectangulo(4.0, 6.0);
        Rectangulo r2 = new Rectangulo(5.0, 5.0);

        r1.mostrar();
        r2.mostrar();
    }
}
```

Resultado en consola:

```
Problems 1   Debug Console   Output   Terminal   Ports
PS C:\Users\VICTUS\OneDrive\guiaape4>  & 'C:\Users\VICTUS\.vscode\extensions\redhat.java-1.58.0-win32
-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\User
s\VICTUS\AppData\Roaming\Code\User\workspaceStorage\bbff34f8966407b46a0f2b3e9759bbb4\redhat.java\jdt_
ws\guiaape4_6f488561\bin' 'Main2'
Rectángulo de 4.0 x 6.0
Área: 24.0
Perímetro: 20.0
¿Es cuadrado? No
---------------------------
Rectángulo de 5.0 x 5.0
Área: 25.0
Perímetro: 20.0
```

Ejercicio 3: Producto y Venta

Producto controla su stock con reducirStock(), que solo actúa si hay unidades suficientes. Venta recibe un objeto Producto como parámetro en aplicar(), reduce el stock y muestra el total a pagar.

- Se probó una venta válida y otra con stock insuficiente.

Producto.java

```
public class Producto {
    String nombre;
    double precio;
    int stock;

    public Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    // Solo reduce el stock si hay unidades suficientes
    public void reducirStock(int cantidad) {
        if (cantidad <= stock) {
            stock -= cantidad;
        } else {
            System.out.println("Stock insuficiente.");
        }
    }

    public void mostrarProducto() {
        System.out.println("Producto: " + nombre + " | Precio: $" +
precio
                + " | Stock: " + stock);
    }
}
```

Venta.java

```
public class Venta {
    int cantidad;

    public Venta(int cantidad) {
        this.cantidad = cantidad;
    }

    // Recibe un objeto Producto como parámetro
    public void aplicar(Producto producto) {
        int stockAntes = producto.stock;
        producto.reducirStock(cantidad);
```

```
        // Si el stock bajó, la venta se realizó
        if (producto.stock < stockAntes) {
            double total = cantidad * producto.precio;
            System.out.println("Venta realizada. Total a pagar: $" +
total);
        }
    }
}
```

Main.java

```
public class Main3 {
    public static void main(String[] args) {
        Producto p = new Producto("Laptop", 750.0, 5);
        p.mostrarProducto();

        System.out.println("\n--- Venta de 2 unidades ---");
        new Venta(2).aplicar(p);
        p.mostrarProducto();

        System.out.println("\n--- Venta de 10 unidades ---");
        new Venta(10).aplicar(p);
        p.mostrarProducto();
    }
}
```

Resultado en consola:

```
Problems 1  Debug Console  Output  Terminal  Ports
\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\VICTUS\AppData\Roaming\Code\User\workspaceStorage\bbff3
4f8966407b46a0f2b3e9759bbb4\redhat.java\jdt_ws\guiaape4_6f488561\bin' 'Main3'
Producto: Laptop | Precio: $750.0 | Stock: 5

--- Venta de 2 unidades ---
Venta realizada. Total a pagar: $1500.0
Producto: Laptop | Precio: $750.0 | Stock: 3

--- Venta de 10 unidades ---
Stock insuficiente.
Producto: Laptop | Precio: $750.0 | Stock: 3
PS C:\Users\VICTUS\OneDrive\guiaape4>
```

Ejercicio 4: Transferencias

Como el enunciado remite a la clase Cuenta del informe, se implementó con titular, saldo, depositar() y retirar(), y se agregó transferir(Cuenta destino, double monto): retira el monto de la cuenta actual y lo deposita en la cuenta destino solo si hay fondos.

- Se hizo una transferencia válida y otra que falla.

Cuenta.java

```
public class Cuenta {
    String titular;
    double saldo;

    public Cuenta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    public void depositar(double monto) {
        if (monto > 0) {
            saldo += monto;
        }
    }

    // Retorna true si el retiro se pudo realizar
    public boolean retirar(double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            return true;
        }
        return false;
    }

    // Recibe otro objeto Cuenta como parámetro
    public void transferir(Cuenta destino, double monto) {
        if (retirar(monto)) {
            destino.depositar(monto);
            System.out.println("Transferencia exitosa de $" + monto
                    + " de " + titular + " a " + destino.titular);
        } else {
            System.out.println("Transferencia fallida: fondos "
                    + "insuficientes en la cuenta de " + titular);
        }
    }

    public void mostrar() {
        System.out.println("Titular: " + titular + " | Saldo: $" +
saldo);
```

```
    }
}
```

Main.java

```
public class Main4 {
    public static void main(String[] args) {
        Cuenta c1 = new Cuenta("Ana", 500.0);
        Cuenta c2 = new Cuenta("Luis", 200.0);

        System.out.println("--- Estado inicial ---");
        c1.mostrar();
        c2.mostrar();

        System.out.println("\n--- Transferencia válida ---");
        c1.transferir(c2, 150.0);

        System.out.println("\n--- Transferencia que falla ---");
        c1.transferir(c2, 1000.0);

        System.out.println("\n--- Estado final ---");
        c1.mostrar();
        c2.mostrar();
    }
}
```

Resultado en consola:

```
Problems 1  Debug Console  Output  Terminal  Ports

 ws\guiaape4_6f488561\bin' 'Main4'
 --- Estado inicial ---
 Titular: Ana | Saldo: $500.0
 Titular: Luis | Saldo: $200.0

 --- Transferencia válida ---
 Transferencia exitosa de $150.0 de Ana a Luis

 --- Transferencia que falla ---
 Transferencia fallida: fondos insuficientes en la cuenta de Ana

 --- Estado final ---
 Titular: Ana | Saldo: $350.0
 Titular: Luis | Saldo: $350.0
 PS C:\Users\VICTUS\OneDrive\guiaape4>
```

Ejercicio 5: Biblioteca

Usuario, Libro y Prestamo colaboran entre sí. Prestamo.registrar() llama a Libro.prestar(), que cambia la disponibilidad del libro y devuelve true o false, y muestra un mensaje según el resultado.

- Dos usuarios piden el mismo libro: el primero lo obtiene y el segundo no; luego se devuelve el libro con devolver() y el segundo usuario lo vuelve a intentar.

- **Usuario.java**

```
class Usuario {
    String nombre;
    String cedula;

    Usuario(String nombre, String cedula) {
        this.nombre = nombre;
        this.cedula = cedula;
    }
}

class Libro {
    String titulo;
    boolean disponible = true;

    Libro(String titulo) {
        this.titulo = titulo;
    }

    boolean prestar() {
        if (disponible == true) {
            disponible = false;
            return true;
        } else {
            return false;
        }
    }

    void devolver() {
        disponible = true;
    }
}

class Prestamo {
    Usuario usuario;
    Libro libro;

    Prestamo(Usuario usuario, Libro libro) {
```

```
        this.usuario = usuario;
        this.libro = libro;
    }

    void registrar() {
        boolean resultado = libro.prestar();
        if (resultado == true) {
            System.out.println("Prestamo exitoso: " + usuario.nombre
                    + " se llevo el libro " + libro.titulo);
        } else {
            System.out.println("Prestamo fallido: " + usuario.nombre
                    + " no pudo llevarse el libro " + libro.titulo
                    + " porque ya esta prestado");
        }
    }
}

public class Main5 {
    public static void main(String[] args) {
        Usuario u1 = new Usuario("Maria Perez", "1805123456");
        Usuario u2 = new Usuario("Carlos Gomez", "1804987654");
        Libro libro = new Libro("Don Quijote");

        Prestamo p1 = new Prestamo(u1, libro);
        Prestamo p2 = new Prestamo(u2, libro);

        p1.registrar();
        p2.registrar();

        libro.devolver();
        System.out.println("El libro fue devuelto");
        p2.registrar();
    }
}
```

Resultado en consola:

```
Problems 4  Debug Console  Output  Terminal  Ports

Prestamo exitoso: Maria Perez se llevo el libro Don Quijote
Prestamo fallido: Carlos Gomez no pudo llevarse el libro Don Quijote porque ya esta prestado
El libro fue devuelto
Prestamo exitoso: Carlos Gomez se llevo el libro Don Quijote
PS C:\Users\VICTUS\OneDrive\guiaape4>
```

2.8 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☒ Trabajo en equipo
☐ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

2.9 Conclusiones

- La programación orientada a objetos permite organizar el código en clases con atributos y métodos, lo que disminuye los errores y facilita la reutilización del código.

- Usar objetos como atributos (Libro con Autor; Prestamo con Usuario y Libro) y como parámetros de métodos (Venta con Producto; Cuenta con otra Cuenta) permite que las clases colaboren sin repetir lógica.

- Validar las condiciones dentro de los métodos (stock, fondos y disponibilidad) mantiene consistente el estado de los objetos y evita operaciones inválidas.

2.10 Recomendaciones

Antes de elaborar las tareas, revisar el material proporcionado y la bibliografía.
Si se tienen dudas sobre el tema de la APE, acudir a las tutorías académicas.
Comentar el código y probar cada método con casos válidos y no válidos.

2.11 Referencias bibliográficas

[1] M. D. Contreras Castro, *Lógica de programación a partir del pensamiento lógico con software DFD, PSeInt y LPP*, 1.ª ed. Valencia, España: Ediciones USTA, 2023.

[2] M. I. Barquilla Galeano, *Java programación: teoría y ejemplos*, 1.ª ed. Madrid, España: RA-MA Editorial, 2025.

2.12 Anexos

Anexo 1. Proyecto Java para Visual Studio Code (carpeta APE4_VSCode, con los paquetes ejercicio1 a ejercicio5).

Anexo 2. Capturas de la ejecución de cada ejercicio en Visual Studio Code.

Figura A1. Ejecución del Ejercicio 1 (Libro y Autor) en Visual Studio Code.

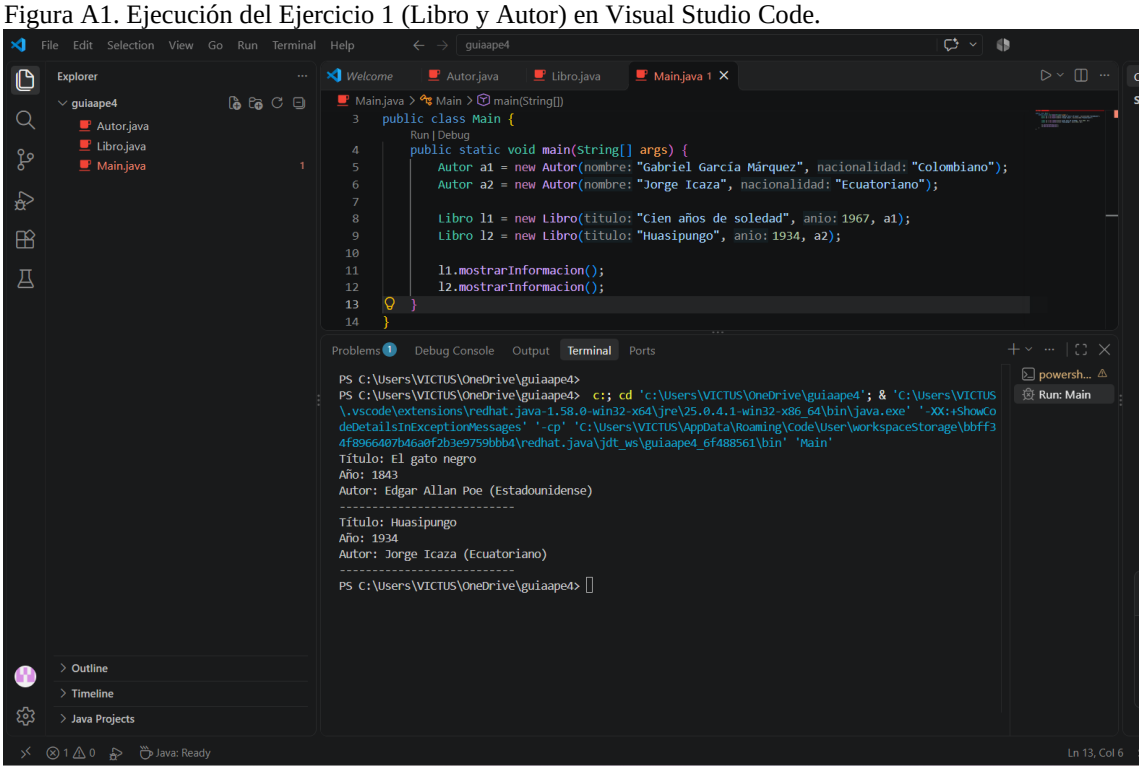


Figura A2. Ejecución del Ejercicio 2 (Rectángulo) en Visual Studio Code.



Figura A3. Ejecución del Ejercicio 3 (Producto y Venta) en Visual Studio Code.

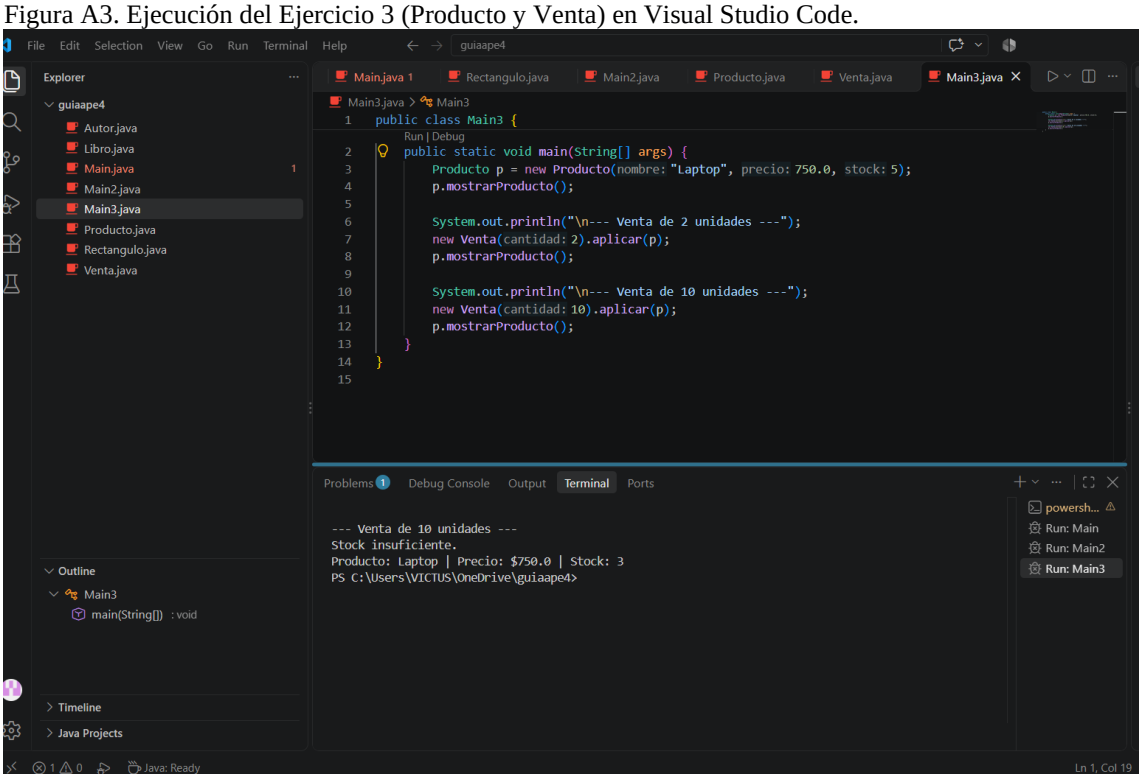


Figura A4. Ejecución del Ejercicio 4 (Transferencias) en Visual Studio Code.



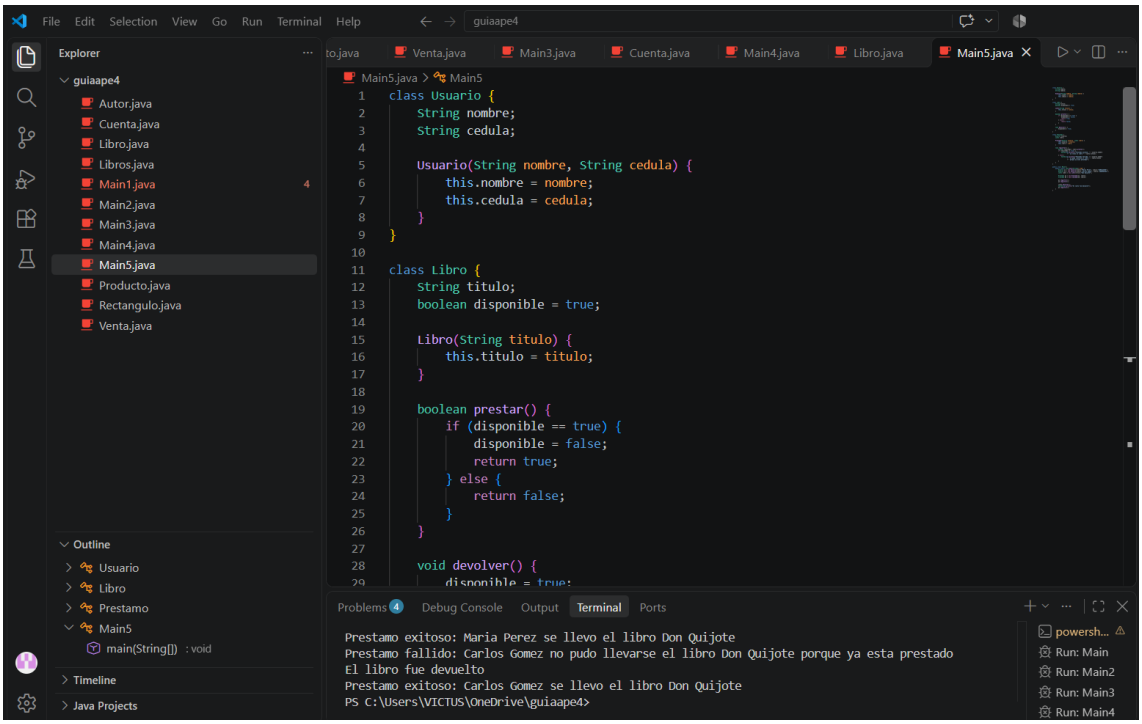
Figura A5. Ejecución del Ejercicio 5 (Biblioteca) en Visual Studio Code.

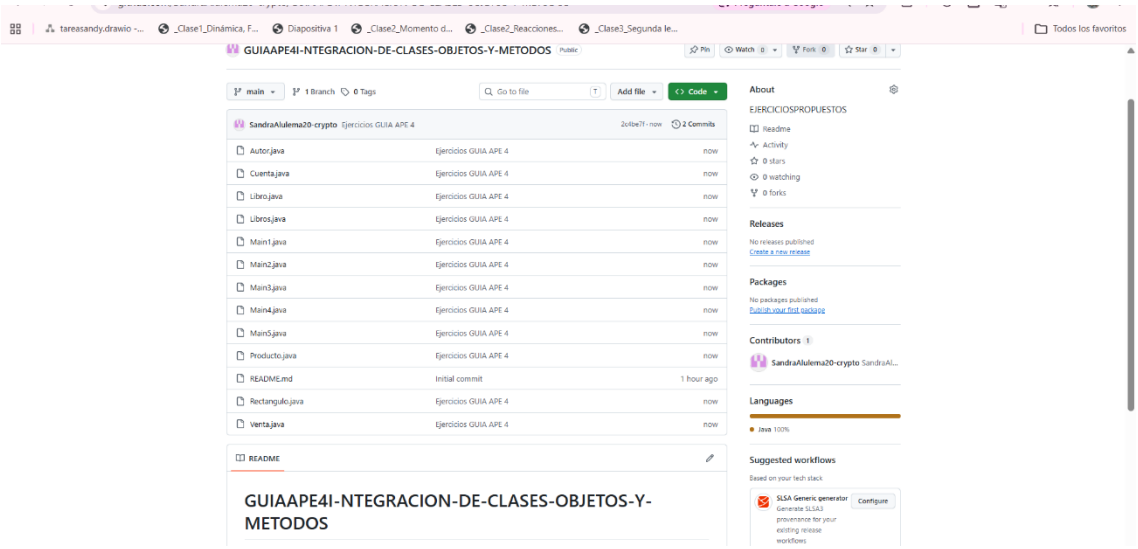
UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO – DICIEMBRE 2026





GITHUB LINK.;